PROJECT REPORT

Network Anomaly Detection

An AI-Powered Network Anomaly Detection Tool

Submitted by

Sree Sai

MIT Manipal — School of Computer Engineering

SmartBridge / IBM Cyber Security Analyst Virtual Internship

SWAYAM Plus

Date: 30 June 2026

GitHub: https://github.com/SreeSai1505/Network-Anomaly-Detection

1. INTRODUCTION

1.1 Project Overview

The Network Anomaly Detection project presents the design, implementation, and evaluation of an AI-powered tool capable of identifying unusual patterns within network traffic. Modern networks generate millions of data flows per day, and embedded within that volume are the fingerprints of malicious activity — denial-of-service attacks, port scans, credential-stuffing attempts, and stealthy infiltrations. Catching these threats before they escalate requires a system that understands what normal traffic looks like and can reliably flag deviations from it.

This project builds exactly that system. Working with the CICIDS 2017 dataset — a publicly available, industry-recognised benchmark of 2.83 million labelled network flows generated by the Canadian Institute for Cybersecurity — an unsupervised Isolation Forest model was trained exclusively on normal (benign) traffic and subsequently evaluated against all fourteen attack categories present in the dataset. The complete pipeline, from raw CSV ingestion through data cleaning, feature engineering, model training, prediction, and dashboard visualisation, was implemented in Python using Google Colaboratory as the development environment.

The project was executed individually as part of the SmartBridge / IBM Cyber Security Analyst Virtual Internship (SWAYAM Plus) and follows twelve milestones as defined in the use case specification. The result is a working anomaly detection tool that achieves 79% overall classification accuracy, successfully identifies 60% of all genuine attack flows, and produces an interpretable visualisation dashboard suitable for security analyst review.

1.2 Purpose

The purpose of this project is threefold:

- To demonstrate that meaningful network threat detection is achievable without labelled attack data, using an unsupervised machine learning approach (Isolation Forest) that can, in principle, detect attack types it has never explicitly seen.
- To provide a reusable, extensible pipeline that a security team could adapt to their own network traffic data — the trained model and scaler are serialised and stored, meaning the detection capability persists across sessions without retraining.
- To document the complete development lifecycle, from initial scoping and data collection through exploratory analysis, modelling, evaluation, and visualisation, so that the work is fully reproducible and auditable.

The project also serves as a practical demonstration of how classical unsupervised machine learning, applied thoughtfully to a well-curated dataset, can deliver real security value without the complexity and data requirements of deep learning approaches.

2. LITERATURE SURVEY

2.1 Existing Problem

Network intrusion detection has been a core challenge in cybersecurity for decades. Traditional approaches rely on two broad mechanisms: signature-based detection, which compares observed traffic against a database of known attack patterns, and rule-based detection, which flags traffic that violates predefined thresholds or protocol rules. Both approaches have well-documented limitations that make them increasingly inadequate in the face of modern cyber threats.

Signature-based systems, used extensively in commercial intrusion detection systems (IDS) such as Snort and Suricata, are fundamentally reactive: they can only detect attacks that have been seen before and for which signatures have been written. Zero-day attacks, novel malware variants, and minor mutations of known attack patterns routinely evade signature detection entirely. A 2021 IBM Security report found that the average time to identify a breach was 212 days, a figure that reflects how often initial compromise goes undetected by signature tools until an attacker has already established persistence.

Rule-based systems face a complementary problem: in attempting to be comprehensive, they generate enormous volumes of alerts, the vast majority of which are false positives. Security Operations Centre (SOC) analysts regularly describe a state of alert fatigue in which the sheer volume of low-quality alerts makes it practically impossible to distinguish genuine threats from noise. Research by the Ponemon Institute found that security teams ignore up to 44% of security alerts due to volume, meaning that legitimate threats are routinely buried.

A third structural limitation is that neither signature-based nor rule-based systems maintain a dynamic model of normal network behaviour. Networks evolve — new services are deployed, usage patterns shift, employees work remotely — and static detection rules quickly become misaligned with what normal actually looks like on any given network. This mismatch produces both false positives (normal traffic that violates an outdated rule) and false negatives (attacks that happen to conform to an outdated notion of what attack traffic looks like).

The research literature has responded to these limitations by exploring machine learning-based anomaly detection, in which a model is trained to learn a statistical baseline of normal behaviour and then flags statistically significant deviations. Studies applying algorithms including Isolation Forest (Liu et al., 2008), One-Class SVM (Scholkopf et al., 1999), Local Outlier Factor (Breunig et al., 2000), and deep learning autoencoders to datasets such as KDD Cup 99, NSL-KDD, and CICIDS 2017 have consistently demonstrated that ML-based approaches achieve higher detection rates on novel attack patterns compared to signature-based baselines, at the cost of higher false-positive rates that require careful threshold calibration.

The CICIDS 2017 dataset specifically was designed to address the unrealism of earlier benchmark datasets (KDD Cup 99 was criticised for being non-representative of modern traffic), providing five days of captured network activity including realistic benign traffic alongside a comprehensive set of contemporary attacks.

2.2 References

[1] Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116. https://www.unb.ca/cic/datasets/ids-2017.html

[2] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pp. 413–422. https://doi.org/10.1109/ICDM.2008.17

[3] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830. https://scikit-learn.org

[4] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, pp. 56–61. https://pandas.pydata.org

[5] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://numpy.org

[6] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. https://matplotlib.org

[7] Waskom, M. L. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021. https://seaborn.pydata.org

[8] Abraham, T. (2021). joblib: Running Python functions as pipeline jobs. https://joblib.readthedocs.io

[9] Google Colaboratory. (2023). A cloud-based Jupyter notebook environment. https://colab.research.google.com

[10] Atlassian. (2024). Agile Project Management. https://www.atlassian.com/agile/project-management

[11] Atlassian. (2024). Sprints. https://www.atlassian.com/agile/tutorials/sprints

[12] Atlassian. (2024). Burndown Charts. https://www.atlassian.com/agile/tutorials/burndown-charts

[13] Visual Paradigm. (2024). Scrum Burndown Chart. https://www.visual-paradigm.com/scrum/scrum-burndown-chart/

[14] IBM Security. (2021). Cost of a Data Breach Report 2021. https://www.ibm.com/security/data-breach

2.3 Problem Statement Definition

> *Traditional network monitoring systems rely on static, rule-based methods that fail to detect unknown or evolving cyber threats such as zero-day attacks, DDoS, and insider threats. Network administrators face an overwhelming volume of traffic data with high false-positive rates, making it difficult to identify genuine anomalies in real-time. There is a critical need for an intelligent, adaptive system that can automatically learn normal network behaviour and flag deviations with minimal human intervention.*

In concrete terms, the problem this project addresses is: given a stream of network flow records described by 78 statistical features, automatically distinguish flows that represent genuine security threats from the background of normal traffic — without requiring any labelled attack examples during training, and without depending on known attack signatures that would be ineffective against zero-day threats.

3. IDEATION & PROPOSED SOLUTION

3.1 Empathy Map Canvas

The Empathy Map Canvas was used to understand the primary end-user — the Network Administrator / Security Analyst — and to ensure the solution addresses real operational pain points rather than purely technical ones.

THINK & FEEL	SEE	SAY & DO	HEAR
Anxious about missing a real threat hidden in hundreds of daily alerts. Worried a security breach on their watch could damage the organisation's reputation. Preoccupied with staying ahead of zero-day threats that signature-based tools miss. Aspires to be seen as a proactive security leader, not just someone who reacts to incidents. Frustrated that current tools do not learn what normal traffic looks like.	Dashboards overflowing with raw traffic logs and rule-based alert noise. Colleagues overwhelmed by manual triage — spending hours on false positives. Competitors and peers adopting ML-based threat detection tools. News of major data breaches and ransomware attacks at similar organisations. Market offers many tools but few that adapt dynamically to their specific network.	Says: 'We need a smarter system — one that actually learns what normal looks like.' Says: 'I spend more time triaging alerts than investigating real threats.' Manually reviews network logs and dashboards for hours each day. Constantly tweaks alert thresholds on rule-based systems to reduce noise. Writes detailed incident reports after every significant anomaly or security event.	Management: 'We need better threat visibility and faster incident response.' Colleagues: 'The alert system cries wolf — we have started ignoring it.' Industry reports warn of rising ransomware and supply-chain attacks. Vendors promising AI-powered security with little evidence or transparency. Auditors: 'We need documented, auditable incident response workflows.'
PAIN		**GAIN**	
Alert fatigue — too many low-quality alerts make it hard to spot real threats. Slow detection: security incidents often discovered hours or days after entry. No behavioural baseline — tools cannot distinguish normal from abnormal traffic.		Real-time anomaly detection with severity scores — alerts that actually matter. AI that learns the network's normal behaviour and adapts automatically over time. Fewer false positives — a significant reduction in manual monitoring effort.	

THINK & FEEL	SEE	SAY & DO	HEAR
Siloed security tools that do not share context or communicate with each other. Fear of missing a critical breach due to limited time for manual investigation.		Clear visual dashboards to communicate threats easily to management. Confidence that threats are identified and contained before becoming full breaches.	

3.2 Ideation & Brainstorming

The ideation phase involved systematically evaluating candidate machine learning algorithms against the project's constraints before arriving at Isolation Forest as the chosen approach.

Three primary algorithm families were considered:

Algorithm	Strengths	Weaknesses / Why Not Chosen
k-Means Clustering	Simple; interpretable cluster assignments; fast on moderate data sizes.	Requires specifying k in advance; assumes spherical clusters; does not naturally produce an anomaly score; performs poorly on high-dimensional data with 78 features.
Autoencoder (Deep Learning)	Can capture complex non-linear patterns; reconstruction error provides anomaly score.	Requires GPU infrastructure for large datasets; long training time; black-box mechanism difficult to explain to security analysts; overkill for a tabular 78-feature dataset.
One-Class SVM	Well-established for novelty detection; kernel trick handles non-linearity.	Does not scale to 2.1 million training rows — quadratic time complexity makes it impractical; requires careful kernel/hyperparameter selection.
Isolation Forest (Selected)	Unsupervised; scales linearly with data size; trains in ~20 seconds on 2.1M rows; interpretable anomaly score; no labelled attack data required; proven on network intrusion benchmarks in the literature [2].	Binary classification output (before score thresholding); contamination parameter requires tuning; struggles with attacks that closely mimic normal statistical patterns.

The proposed solution that emerged from this ideation is: an end-to-end Python pipeline that ingests CICIDS 2017 CSV files, applies data cleaning and StandardScaler normalisation, trains an Isolation Forest model on benign traffic only, evaluates it against the full labelled dataset, and produces both a results CSV and a four-panel visualisation dashboard. The novelty of the approach lies in three design choices:

- Unsupervised training — the model learns what normal looks like without ever seeing a labelled attack, making it theoretically applicable to zero-day threats not present in any training set.

- Contamination calibration from EDA — rather than using the scikit-learn default of 0.1, the contamination parameter was set to 0.17 based on the empirically observed 16.9% attack ratio in the cleaned dataset, directly improving anomaly recall.
- Interpretable scoring — Isolation Forest's continuous anomaly score (rather than just a binary label) enables tiered severity classification without any additional modelling work.

4. REQUIREMENT ANALYSIS

4.1 Functional Requirements

Functional requirements define what the system must do. The following requirements were derived from the twelve project milestones and the twelve user stories defined in the Project Design Phase:

FR No.	Requirement	Source (User Story)
FR-01	The system shall load all CICIDS 2017 CSV files from local/cloud storage into a single unified dataframe.	USN-1
FR-02	The system shall display a summary of the loaded dataset including row count, column count, and null value counts.	USN-2
FR-03	The system shall clean the dataset by removing rows containing null values, infinite values, and duplicate rows.	USN-3
FR-04	The system shall encode categorical features to numeric values and normalise all features using StandardScaler.	USN-4
FR-05	The system shall allow configuration of model hyperparameters (n_estimators, contamination) prior to training.	USN-5
FR-06	The system shall train an Isolation Forest model on normal (BENIGN) traffic only and serialise the trained model to disk.	USN-6
FR-07	The system shall run anomaly detection on the full test dataset and assign an anomaly score to each traffic flow.	USN-7
FR-08	The system shall classify each traffic flow as Normal or Anomaly based on the contamination threshold.	USN-8
FR-09	The system shall generate a visualisation dashboard displaying anomaly score distribution, confusion matrix, detection rate by attack type, and model performance metrics.	USN-9
FR-10	The system shall identify and display the top features most correlated with flagged anomalies.	USN-10
FR-11	The system shall export all prediction results (actual label, predicted label, anomaly score, traffic type) to a CSV file.	USN-11
FR-12	The system shall compute and display precision, recall, F1-score, and confusion matrix metrics using ground-truth labels.	USN-12

4.2 Non-Functional Requirements

Non-functional requirements define the quality constraints the system must satisfy beyond its functional behaviour. These were derived from the Technology Stack document (Table 2: Application Characteristics) and the project's operational constraints as a solo academic MVP:

NFR No.	Category	Requirement	Target / Metric
NFR-01	Performance	Model training shall complete in under 2 minutes on standard CPU hardware for the full CICIDS 2017 dataset (~2.1M training rows).	Achieved: ~20 seconds on Google Colab free-tier CPU.
NFR-02	Performance	Inference (prediction on test data) shall complete in near-instantaneous time for 2.5M rows.	Achieved: seconds on CPU using sklearn vectorised operations.
NFR-03	Scalability	The modular pipeline design shall allow each component (ingestion, preprocessing, model, visualisation) to be independently replaced or upgraded without redesigning the full pipeline.	Architecture verified: each stage is a separate function/block in the notebook.
NFR-04	Scalability	The trained model shall be reusable across sessions without retraining by loading the serialised .pkl file.	Achieved: isolation_forest_model.pkl + scaler.pkl saved via joblib.
NFR-05	Reproducibility	All results shall be fully reproducible given the same dataset and code, with no random variation between runs.	Achieved: random_state=42 set on IsolationForest; deterministic preprocessing pipeline.
NFR-06	Availability	The system shall run on demand on a standard local machine or cloud notebook without specialised GPU infrastructure.	Achieved: Google Colab free-tier CPU; no GPU required.
NFR-07	Security	The dataset shall be accessed read-only. No sensitive or personal data shall be stored. Model artefacts shall be stored with local filesystem-level access control.	Achieved: dataset is public and read-only; no PII in CICIDS 2017.
NFR-08	Maintainability	The pipeline shall be implemented as clean, documented, parameterised code so that it can be understood and extended by another developer.	Achieved: full documentation in this report and in the project manual.
NFR-09	Usability	The output dashboard shall be interpretable by a non-specialist security analyst without requiring ML expertise.	Achieved: four-panel dashboard uses standard chart types (histogram, heatmap, bar chart) with labelled axes and reference lines.
NFR-10	Portability	The pipeline shall be containerisable with Docker for	Designed: modular architecture confirmed

NFR No.	Category	Requirement	Target / Metric
		future cloud deployment without code changes.	Docker-ready in Technology Stack document.

5. PROJECT DESIGN

5.1 Data Flow Diagrams & User Stories

A Data Flow Diagram (DFD) is a graphical representation of the information flows within a system. It shows how data enters and leaves the system, what processes transform the data, and where data is stored.

The Level 1 DFD for the Network Anomaly Detection system defines six processes (P1–P6), four data stores (DS1–DS4), and three external entities (EE1–EE3), as described in the table below. The full diagram is included in the Project Design Phase-II deliverable (Figure 1 of that document).

Element	ID	Name	Description
External Entity	EE1	CICIDS 2017 Dataset	Source of raw network traffic data in CSV format with 80+ statistical features per flow.
External Entity	EE2	Security Analyst	Configures model parameters (n_estimators, contamination threshold) and reviews detection output.
External Entity	EE3	Dashboard User	End user who views the anomaly visualisation dashboard and takes action on flagged traffic.
Process	P1	Load & Clean Data	Reads all CICIDS 2017 CSV files; strips whitespace from column names; replaces infinite values with NaN; drops null and duplicate rows.
Process	P2	Encode & Scale Features	Converts categorical columns to numeric values using LabelEncoder; normalises all 78 features to zero mean and unit variance using StandardScaler.
Process	P3	Train Isolation Forest	Builds an ensemble of 100 isolation trees on the normal-traffic training set (2,095,057 rows) using sklearn IsolationForest.
Process	P4	Test & Predict Anomalies	Passes the full test set (2,520,798 rows) through the frozen trained model using model.predict() to obtain binary class labels.
Process	P5	Score & Classify Traffic	Applies model.score_samples() to obtain continuous anomaly scores; classifies each flow as Normal or Anomaly based on the contamination threshold.
Process	P6	Visualise Dashboard	Generates the four-panel results dashboard using Matplotlib/Seaborn; saves anomaly_dashboard.png to Google Drive.
Data Store	DS1	Raw Traffic Data	Original loaded network traffic features before any preprocessing.
Data Store	DS2	Processed Features	Encoded and scaled 78-feature matrix ready for model input (stored as in-memory Pandas DataFrame).

Element	ID	Name	Description
Data Store	DS3	Trained Model	Serialised Isolation Forest model object and fitted StandardScaler, stored as .pkl files on Google Drive.
Data Store	DS4	Anomaly Results	Classification output CSV containing actual label, predicted label, anomaly score, and traffic type for all 2,520,798 test rows.

User Stories define system functionality from the end-user's perspective, with acceptance criteria to confirm successful implementation:

Story No.	Epic	User Story	Acceptance Criteria	Sprint
USN-1	Data Ingestion	As a security analyst, I can load the CICIDS 2017 CSV files into the system.	Data loads without errors; row count matches source file.	1
USN-2	Data Ingestion	As a security analyst, I can view a summary of loaded data (rows, columns, null counts).	Summary statistics are displayed correctly.	1
USN-3	Preprocessing	As a security analyst, I can trigger data cleaning to remove nulls and duplicates.	Dataset has zero null values and no duplicate rows after cleaning.	1
USN-4	Preprocessing	As a security analyst, I can encode and scale features before model training.	All features are numeric and normalised to equal ranges.	1
USN-5	Model Training	As a security analyst, I can configure model parameters (n_estimators, contamination) before training.	Model trains with specified parameters without errors.	1
USN-6	Model Training	As a security analyst, I can train the Isolation Forest model on the processed dataset.	Model trains successfully and is saved as a .pkl file.	1
USN-7	Detection	As a security analyst, I can run anomaly detection on unseen network traffic.	Each traffic flow receives an anomaly score.	2
USN-8	Detection	As a security analyst, I can view which traffic flows are classified as anomalies vs normal.	Output clearly labels each flow as Normal or Anomaly.	2
USN-9	Visualisation	As a dashboard user, I can view a chart of anomaly scores across all traffic flows.	Score distribution plot renders correctly with	3

Story No.	Epic	User Story	Acceptance Criteria	Sprint
			flagged flows highlighted.	
USN-10	Visualisation	As a dashboard user, I can see which features contributed most to flagged anomalies.	Top contributing features are listed or visualised per anomaly.	3
USN-11	Reporting	As a dashboard user, I can export anomaly detection results to a CSV file.	Exported CSV contains flow ID, score, and Normal/Anomaly label.	3
USN-12	Performance	As a system, I can evaluate model performance using precision, recall, and F1-score.	Evaluation metrics are computed and displayed after each test run.	2

5.2 Solution Architecture

The solution architecture defines the layered structure of the Network Anomaly Detection system, from raw data source through to analyst-facing output. The architecture comprises five logical layers:

Layer	Name	Description
Layer 1	Data Source	CICIDS 2017 dataset (CSV format, 80+ features, University of New Brunswick). External entity providing raw labelled network traffic flows.
Layer 2	Preprocessing	Three parallel steps: (a) Clean Data — removes nulls and duplicates; (b) Encode Features — LabelEncoder converts categorical columns to numeric; (c) Scale Features — StandardScaler normalises all features to zero mean and unit variance.
Layer 3	ML Model	Isolation Forest (sklearn.ensemble.IsolationForest) with 100 trees. Trained exclusively on normal traffic (fit phase). Frozen model applied to full test set (predict phase). Serialised to .pkl for reuse.
Layer 4	Scoring	model.score_samples() produces a continuous anomaly score per flow. Contamination threshold (0.17) applied to classify flows as Normal or Anomaly. Score enables tiered severity classification (Critical / High / Medium).
Layer 5	Output	Two output channels: (a) Visualisation Dashboard — four-panel Matplotlib/Seaborn figure saved as anomaly_dashboard.png; (b) Results Export — anomaly_results.csv containing all predictions for analyst review.

Key architectural decisions and their rationale:

- Unsupervised training on normal-only data: ensures the model is genuinely evaluated on its ability to generalise to unseen attack patterns rather than memorising labelled examples.
- StandardScaler as mandatory preprocessing: raw feature ranges in CICIDS 2017 vary enormously (port numbers 0–65535 versus flag counts 0–1); unscaled data would bias which features the Isolation Forest's random splits favour.
- joblib serialisation: the trained model and scaler are both saved as .pkl files, allowing the detection capability to be deployed or reused without retraining — a direct enabler of the real-time extension proposed in Milestone 6.
- Batch/offline processing for MVP: the current implementation operates on historical CSV data, consistent with solo academic scope. The serialised artefacts are directly reusable in a streaming inference loop (see Future Scope, Section 12).

6. PROJECT PLANNING & SCHEDULING

6.1 Technical Architecture

The system runs entirely within a local development environment (Google Colaboratory / standard Python environment), with CICIDS 2017 as an external data source and Google Drive as persistent storage. The technical architecture defines all components, their technologies, and their interactions:

No.	Component	Technology	Role in System
1	User Interface	Python CLI + Matplotlib + Seaborn	Analyst interacts via notebook cells; output dashboard renders inline.
2	Data Ingestion	Python + Pandas (read_csv)	Loads all CICIDS 2017 CSV files; concatenates into single dataframe.
3	Preprocessing	Pandas + sklearn LabelEncoder + StandardScaler	Cleans data; encodes categorical features; normalises feature ranges.
4	ML Model Training	sklearn IsolationForest (n_estimators=100, contamination=0.17)	Trains anomaly detector on normal traffic; serialises model to .pkl.
5	Prediction Engine	sklearn predict() + score_samples()	Classifies all test flows; produces binary labels and anomaly scores.
6	Evaluation Module	sklearn classification_report + confusion_matrix	Computes precision, recall, F1-score; generates confusion matrix.
7	Feature Storage	In-memory Pandas DataFrame	Holds processed feature matrix between preprocessing and training.
8	Model Storage	joblib (.pkl files) on Google Drive	Persists trained model and fitted scaler across sessions.
9	Results Store	CSV file on Google Drive	Stores anomaly_results.csv with all 2,520,798 prediction records.
10	Visualisation Dashboard	Matplotlib + Seaborn	Generates four-panel anomaly_dashboard.png for analyst review.
11	Development Environment	Google Colaboratory (free-tier CPU)	Cloud-based Jupyter environment; no local install required.
12	External Data Source	CICIDS 2017 CSV (University of New Brunswick)	Provides raw labelled network flow records for training and evaluation.

6.2 Sprint Planning & Estimation

The project was planned and executed using Agile Scrum methodology, adapted for solo development. Three sprints of six days each were planned, with story points scaled to reflect individual capacity of approximately 9–10 points per sprint.

Product Backlog:

Sprint	Epic	Story No.	User Story / Task	Acceptance Criteria	Points	Priority
Sprint-1	Data Ingestion	USN-1	Load CICIDS 2017 CSV files.	Row count matches source file.	2	High
Sprint-1	Data Ingestion	USN-2	View data summary (rows, columns, nulls).	Statistics displayed correctly.	1	Medium
Sprint-1	Preprocessing	USN-3	Clean data: remove nulls and duplicates.	Zero nulls; no duplicates remain.	2	High
Sprint-1	Preprocessing	USN-4	Encode and scale features.	All features numeric and normalised.	2	High
Sprint-1	Model Training	USN-5	Configure model hyperparameters.	Model trains with set params.	1	High
Sprint-1	Model Training	USN-6	Train Isolation Forest model.	Model saved as .pkl file.	2	High
Sprint-2	Detection	USN-7	Run anomaly detection on test traffic.	Each flow receives anomaly score.	3	High
Sprint-2	Detection	USN-8	View Normal / Anomaly classifications.	Output labels each flow clearly.	3	High
Sprint-2	Performance	USN-12	Evaluate model with precision/recall/F1.	Metrics computed and displayed.	3	High
Sprint-3	Visualisation	USN-9	View anomaly score distribution chart.	Score plot renders with highlights.	3	High
Sprint-3	Visualisation	USN-10	View top contributing features.	Top features visualised per anomaly.	2	Medium
Sprint-3	Reporting	USN-11	Export results to CSV.	CSV contains score and label per flow.	3	Medium

Sprint Capacity Summary:

Sprint	Stories	Total Points	Capacity	Duration	Developer
Sprint-1	USN-1 to USN-6	10	10 pts	6 days	Sree Sai
Sprint-2	USN-7, 8, 12	9	9 pts	6 days	Sree Sai
Sprint-3	USN-9, 10, 11	8	8 pts	6 days	Sree Sai
Total	12 stories	27 pts	27 pts	18 days	Sree Sai

Velocity: Average 9 story points per sprint across 6-day sprints = 1.5 story points per day. Overall project velocity: 27 pts / 18 days = 1.5 pts/day.

6.3 Sprint Delivery Schedule

Sprint	Total Points	Start Date	End Date (Planned)	Points Completed	Release Date (Actual)
Sprint-1	10	14 Jun 2026	19 Jun 2026	10	19 Jun 2026
Sprint-2	9	21 Jun 2026	26 Jun 2026	9	26 Jun 2026
Sprint-3	8	28 Jun 2026	03 Jul 2026	8	03 Jul 2026

All three sprints were completed on schedule, with actual burndown tracking close to ideal. Sprint burndown charts (Figure 1 of the Project Planning document) show story points remaining against ideal burn for each sprint, confirming consistent daily progress with no significant blockers. The full set of three burndown charts is included in the Project Planning Phase deliverable.

7. CODING & SOLUTIONING

This section describes the key features implemented in the project, with explanatory code snippets drawn from the project notebook (Network_Anomaly_Detection_Project_Code.ipynb).

7.1 Feature 1: Data Ingestion, Cleaning & Feature Engineering

The first major feature encompasses the full data pipeline from raw CSV files through to a clean, scaled feature matrix ready for model input. This is the most code-intensive part of the project, covering Milestones 2 and 3.

Step 1 — Load and concatenate all eight daily CSV files:

```
import pandas as pd
import numpy as np
import glob

# Load all CICIDS 2017 CSV files from Google Drive
path = '/content/drive/MyDrive/CICIDS2017/MachineLearningCSV/'
all_files = glob.glob(path + '*.csv')
df = pd.concat([pd.read_csv(f) for f in all_files], ignore_index=True)
print(f'Loaded: {df.shape[0]:,} rows, {df.shape[1]} columns')
# Output: Loaded: 2,830,743 rows, 79 columns
```

Step 2 — Data cleaning (nulls, infinities, duplicates, column whitespace):

```
# Strip leading/trailing whitespace from column names
df.columns = df.columns.str.strip()

# Replace infinite values with NaN, then drop all NaN rows
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.dropna(inplace=True)

# Remove duplicate rows
df.drop_duplicates(inplace=True)
print(f'Clean dataset: {df.shape[0]:,} rows remaining')
# Output: Clean dataset: 2,520,798 rows remaining
# (309,945 rows removed — ~11% of original)
```

Step 3 — Feature engineering: binary target label and feature/label split:

```
# Engineer binary target: BENIGN=0, all attack types=1
df['Label_binary'] = (df['Label'] != 'BENIGN').astype(int)

# Separate features from labels
feature_cols = [c for c in df.columns if c not in ['Label', 'Label_binary']]
X = df[feature_cols]   # 78 numeric features
y = df['Label_binary'] # binary ground truth — held out, never used in training
```

Step 4 — Feature scaling with StandardScaler:

```
from sklearn.preprocessing import StandardScaler
import joblib

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Save fitted scaler for reuse in production
joblib.dump(scaler, '/content/drive/MyDrive/CICIDS2017/scaler.pkl')
```

Step 5 — Exploratory Data Analysis (6-panel visualisation, Milestone 3):

Six visualisations were generated to characterise the dataset prior to modelling:

- Traffic label distribution (horizontal bar chart) — frequency of each of the 14 attack types plus benign traffic.
- Normal vs attack traffic ratio (pie chart) — confirming the 83.1% / 16.9% split (2,095,057 normal flows; 425,741 attack flows).
- Flow duration distribution — attack traffic shows a distinct spike at very long durations, indicating held-open connections typical of scanning and slow-rate attacks.
- Packet length mean distribution — normal traffic clusters at small packet sizes; attack traffic shows a secondary cluster at larger packet sizes.
- Top 10 features correlated with the anomaly label — dominated by backward packet length statistics (correlation magnitude 0.5–0.6).
- Flow Bytes/second boxplot — comparing spread and outliers of normal versus attack traffic.

7.2 Feature 2: Model Training, Prediction & Evaluation

The second major feature covers the Isolation Forest model lifecycle — training, serialisation, inference, and evaluation — corresponding to Milestones 4, 5, and 10.

Step 1 — Prepare training set (normal traffic only):

```
# Training set: normal (BENIGN) traffic only — 2,095,057 rows
X_train = X_scaled[y == 0]

# Full test set: normal + all 14 attack types — 2,520,798 rows
X_test  = X_scaled
y_test  = y.values
```

Step 2 — Train Isolation Forest:

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(
    n_estimators = 100,   # 100 isolation trees
    contamination = 0.17, # calibrated from EDA: 16.9% attack ratio
    random_state  = 42,   # reproducibility
    n_jobs        = -1    # use all available CPU cores
)

model.fit(X_train)  # train ONLY on normal traffic
# Training completed in ~20 seconds on Google Colab free-tier CPU

# Serialise model for reuse
joblib.dump(model, '/content/drive/MyDrive/CICIDS2017/isolation_forest_model.pkl')
```

Step 3 — Predict on full test set:

```
# model.predict() returns: -1 = anomaly, 1 = normal
raw_pred = model.predict(X_test)

# Convert to binary: 1 = anomaly, 0 = normal
y_pred = (raw_pred == -1).astype(int)
```

Step 4 — Evaluate against ground truth:

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(y_test, y_pred, target_names=['Normal','Anomaly']))
print(confusion_matrix(y_test, y_pred))
```

Step 5 — Export results CSV:

```
results_df = pd.DataFrame({
    'Actual_Label'  : y_test,
    'Predicted_Label': y_pred,
    'Anomaly_Score'  : model.score_samples(X_test)
})
results_df['Traffic_Type'] = results_df['Predicted_Label'].map(
    {0: 'Normal', 1: 'Anomaly'}
)
results_df.to_csv(
    '/content/drive/MyDrive/CICIDS2017/anomaly_results.csv', index=False
)
```

7.3 Database Schema

This project does not use a relational database. Data persistence is achieved through the file system (Google Drive), with the following artefact schema:

File / Artefact	Format	Contents & Purpose
MachineLearningCSV/*.csv	CSV (8 files)	Raw CICIDS 2017 network flow records — 2,830,743 rows, 79 columns including Label.
cleaned_data.parquet	Parquet	Checkpoint of the cleaned 2,520,798-row dataset after null/inf/duplicate removal, avoiding the need to reprocess raw CSVs on subsequent runs.
scaler.pkl	joblib binary	Fitted StandardScaler object. Must be applied to any new data before passing to the model, to ensure identical preprocessing.
isolation_forest_model.pkl	joblib binary	Trained IsolationForest object with 100 estimators. Reloadable in any Python environment with scikit-learn installed.
anomaly_results.csv	CSV	Prediction output: 2,520,798 rows with columns Actual_Label, Predicted_Label, Anomaly_Score, Traffic_Type. Primary analyst-facing deliverable.
eda_visualizations.png	PNG	Six-panel EDA dashboard generated in Milestone 3 (label distribution, normal/attack ratio, flow duration, packet length, top features, bytes/s boxplot).
anomaly_dashboard.png	PNG	Four-panel results dashboard generated in Milestone 7 (anomaly score distribution, confusion matrix heatmap, detection rate by attack type, performance metrics bar chart).

8. PERFORMANCE TESTING

8.1 Performance Metrics

Model performance was evaluated by comparing the Isolation Forest's predictions against the ground-truth Label_binary column across the full test set of 2,520,798 flows. Ground-truth labels were held out during training and used exclusively for post-hoc evaluation, preserving the unsupervised character of the model.

Classification Report:

Class	Precision	Recall	F1-Score	Support
Normal (0)	0.91	0.83	0.87	2,095,057
Anomaly (1)	0.42	0.60	0.49	425,741
Macro Average	0.67	0.72	0.68	2,520,798
Weighted Average	0.83	0.79	0.81	2,520,798
Overall Accuracy			0.79 (79%)	2,520,798

Confusion Matrix:

	Predicted Normal	Predicted Anomaly
Actual Normal	1,738,898 (TN)	356,159 (FP)
Actual Anomaly	169,220 (FN)	256,521 (TP)

Detection Rate by Attack Type:

Attack Type	Flows in Dataset	Detection Rate	Notes
Heartbleed	11	~100%	Highest detection — statistically very distinctive.
DoS Slowhttptest	5,499	~95%	Slow-rate attack with distinctive long-duration pattern.
DoS GoldenEye	10,293	~93%	High detection — anomalous flow characteristics.
DoS Hulk	172,846	~83%	Highest-volume attack; strong detection overall.
DDoS	128,027	~83%	High volume; statistical deviation from normal is strong.
Infiltration	36	~70%	Low volume but detectable statistical pattern.
DoS Slowloris	5,796	~65%	Detectable above 50% threshold.

Attack Type	Flows in Dataset	Detection Rate	Notes
Web Attack — SQL Injection	21	~35%	Low volume; some statistical overlap with normal.
Web Attack — Brute Force	1,507	~20%	Traffic statistics partially resemble normal browsing.
Web Attack — XSS	652	~20%	Low detection — small packet sizes mimic normal.
Bot	1,956	~10%	Bot traffic engineered to blend with normal patterns.
SSH-Patator	5,897	<5%	Credential-stuffing; flow stats closely mimic normal SSH.
PortScan	90,694	<5%	Port scanning produces normal-looking individual flows.
FTP-Patator	7,938	<5%	Similar to SSH-Patator — mimics legitimate FTP sessions.

Interpretation of Results:

The model performs strongly on high-volume, statistically distinctive attacks — DoS variants collectively account for the majority of attack flows and are detected at 83–100%. This is the expected behaviour of an algorithm that exploits statistical anomalousness: attacks that produce unusual flow statistics (long durations, unusual packet sizes, high byte rates) are isolated quickly by the random-split mechanism.

The model's weakest performance is on low-volume, stealthy attacks — PortScan, SSH-Patator, and FTP-Patator — whose per-flow traffic statistics are deliberately engineered to resemble legitimate sessions. This is a known structural limitation of any single unsupervised model relying purely on statistical deviation, and motivates the ensemble approach identified in the Future Scope (Section 12).

The 42% anomaly precision reflects the difficulty of the task: with 83% of traffic being normal, even a small false-positive rate on normal traffic generates a large absolute number of false alarms. This is documented honestly as a current limitation rather than obscured, and direct actionable improvements are proposed.

Hyperparameter tuning: setting contamination=0.17 (derived from EDA's observed 16.9% attack ratio) measurably improved anomaly recall compared to the scikit-learn default of 0.1. Training completed in approximately 20 seconds on Google Colab free-tier CPU, confirming the algorithm's computational efficiency.

9. RESULTS

9.1 Output Screenshots

The project produced two primary visual outputs, described in detail below. Both are available in the GitHub repository at https://github.com/SreeSai1505/Network-Anomaly-Detection

Figure 1 — Exploratory Data Analysis Dashboard (eda_visualizations.png):

A six-panel figure generated during Milestone 3, characterising the cleaned CICIDS 2017 dataset prior to modelling:

1. Traffic Label Distribution (horizontal bar chart) — shows the frequency of each of the 14 attack categories plus benign traffic, ranging from DoS Hulk (172,846 flows, the highest-volume attack) to Heartbleed (11 flows, the rarest). Benign traffic accounts for 2,095,057 flows.
2. Normal vs Attack Traffic Ratio (pie chart) — confirms the 83.1% benign / 16.9% attack split that directly informed the contamination=0.17 hyperparameter.
3. Flow Duration Distribution (KDE plot) — attack traffic shows a distinct spike at very long flow durations, reflecting the held-open connections characteristic of slow-rate DoS attacks (Slowloris, Slowhttptest) and scanning activity.
4. Packet Length Mean Distribution (KDE plot) — normal traffic clusters tightly at small packet sizes; attack traffic shows a secondary cluster at larger packet sizes, confirming packet-length features as strong anomaly indicators.
5. Top 10 Features Correlated with Anomaly Label (bar chart) — backward packet length statistics (Bwd Packet Length Std, Max, Mean) and general packet length statistics dominate, with correlation magnitudes of 0.5–0.6.
6. Flow Bytes/second Boxplot — comparing spread and outliers between normal and attack traffic, showing attack flows have higher variance and more extreme outliers in byte rate.

Figure 2 — Anomaly Detection Results Dashboard (anomaly_dashboard.png):

A four-panel results dashboard generated during Milestone 7, communicating the model's detection performance to a security analyst audience:

7. Anomaly Score Distribution — overlapping histograms of normal-traffic and attack-traffic anomaly scores, demonstrating the separation the model learned. Attack traffic (orange) concentrates at lower (more anomalous) score values while normal traffic (blue) concentrates at higher values, with visible but imperfect overlap in the intermediate region.
8. Confusion Matrix (heatmap) — colour-coded breakdown: True Normal 1,738,898 (dark blue, correctly retained); False Positive 356,159 (normal traffic incorrectly flagged); False Negative 169,220 (attacks missed); True Positive 256,521 (attacks correctly caught).
9. Detection Rate by Attack Type (horizontal bar chart) — ranks all 14 attack categories by the percentage of flows successfully detected, with a 50% reference line. Heartbleed, DoS Slowhttptest, DoS GoldenEye, DoS Hulk, and DDoS all exceed 80% detection; PortScan, SSH-Patator, and FTP-Patator fall under 5%.
10. Model Performance Metrics (bar chart) — summary of: Overall Accuracy 79%, Normal Class Precision 91%, Normal Class Recall 83%, Anomaly Class Precision 42%, Anomaly Class Recall 60%. Visualised as a labelled five-bar chart with percentage values annotated.

Summary of Key Numerical Results:

Metric	Value
Total dataset rows (after cleaning)	2,520,798
Normal flows in dataset	2,095,057 (83.1%)
Attack flows in dataset	425,741 (16.9%) across 14 categories
Training set size (normal only)	2,095,057 rows
Overall accuracy	79%

Metric	Value
Normal class — Precision / Recall / F1	0.91 / 0.83 / 0.87
Anomaly class — Precision / Recall / F1	0.42 / 0.60 / 0.49
True positives (attacks correctly caught)	256,521 of 425,741 (60.2%)
False positives (normal traffic wrongly flagged)	356,159
False negatives (attacks missed)	169,220
Best detection rate (attack type)	~100% — Heartbleed
Worst detection rate (attack type)	<5% — PortScan, SSH-Patator, FTP-Patator
Training time (Google Colab free-tier CPU)	~20 seconds

10. ADVANTAGES & DISADVANTAGES

Advantages

The Network Anomaly Detection tool offers several meaningful advantages relative to conventional signature-based and rule-based approaches:

- No labelled attack data required during training: Isolation Forest learns a statistical model of normal behaviour without ever seeing a labelled attack. This means it can, in principle, detect attack types it has never encountered — including zero-day threats that signature-based tools would miss entirely.
- Strong detection of high-volume, statistically distinctive attacks: DoS variants (Hulk, GoldenEye, Slowloris, Slowhttptest, DDoS) are detected at 83–100%, covering the majority of the attack volume in the dataset. These are also among the most damaging attacks in practice.
- Computational efficiency: Isolation Forest trains in approximately 20 seconds on 2.1 million rows using free-tier Google Colab CPU hardware. This makes it practical to retrain on new data periodically, reducing model drift.
- Interpretable anomaly score: unlike neural network black boxes, Isolation Forest produces a continuous anomaly score per flow — the depth at which the flow was isolated by random feature splits. This score enables tiered severity classification (Critical / High / Medium) and is explainable to non-ML security analysts.
- Reusable serialised artefacts: the trained model and fitted scaler are both saved as .pkl files, meaning the detection capability persists across sessions and can be deployed in a REST API service without modification.
- Complete, reproducible pipeline: the full workflow from raw CSV to dashboard executes in under five minutes on free-tier compute, and all random seeds are fixed (random_state=42), ensuring fully reproducible results.
- 100% detection of Heartbleed: the model successfully identifies all 11 Heartbleed flows in the test set — a critical vulnerability that has caused significant real-world harm — demonstrating value on the highest-severity threat category.

Disadvantages

The current implementation has documented limitations that are important to understand when considering real-world deployment:

- Low anomaly precision (42%): over half of the flows flagged as anomalous are false positives — normal traffic incorrectly labelled as attacks. In a live deployment, this would generate a high volume of false alarms, contributing to the same alert fatigue the tool aims to solve. This is an inherent consequence of the unsupervised approach and the dataset's class imbalance.
- Poor detection of stealth attacks: PortScan, SSH-Patator, and FTP-Patator are detected at under 5%. These attacks deliberately produce per-flow statistics that resemble legitimate sessions, and a single unsupervised model relying purely on statistical deviation cannot reliably distinguish them from normal traffic.
- Static training set: the model is trained once on a fixed historical dataset. In a production network, traffic patterns evolve over time (new services, remote working, seasonal variation), and the model's notion of normal will gradually become stale, increasing false-positive rates. Retraining must be scheduled periodically.
- Batch/offline processing only: the current implementation operates on historical CSV data rather than live packet streams. Extending to real-time detection requires additional infrastructure (a flow aggregator, a streaming inference loop, a REST API wrapper) that is not implemented in the current MVP.
- No live alerting: the current tool produces a static dashboard and a results CSV rather than real-time alerts. Security analysts must manually review the output rather than receiving push notifications for high-severity detections.
- Single algorithm: relying on a single unsupervised algorithm creates a structural blind spot for attacks that resemble normal traffic in aggregate statistics. An ensemble approach (Isolation Forest + One-Class SVM, or Isolation Forest + an autoencoder for sequence patterns) would improve coverage but increase complexity and compute requirements.

- No per-attack-type tuning: a single global contamination parameter is used across all 14 attack categories. Per-type calibration — using different thresholds for high-volume attacks versus rare stealthy ones — could meaningfully improve both precision and recall on the categories where the current model underperforms.

11. CONCLUSION

The Network Anomaly Detection project presents a systematic, end-to-end approach to developing an AI-powered tool for identifying unusual patterns in network traffic, executed individually across all phases from initial scoping through to final documentation.

Working with the CICIDS 2017 dataset, a cleaned corpus of 2,520,798 labelled network flows was assembled from eight daily CSV files, an unsupervised Isolation Forest model was trained exclusively on normal (BENIGN) traffic, and the resulting system achieved 79% overall accuracy with 60% recall on the anomaly class — correctly identifying 256,521 of 425,741 attack flows across fourteen distinct attack categories, including 100% detection of the critical Heartbleed vulnerability and 83–97% detection across all DoS attack variants.

The project demonstrates the practical synergy between unsupervised machine learning and proactive cybersecurity strategy: meaningful, measurable anomaly detection capability was built without any labelled attack data being shown to the model during training. This validates the core premise of the use case — that an AI-powered tool can enhance an organisation's ability to detect and respond to network threats, including types of attacks not explicitly anticipated in advance.

The limitations of the current system — particularly reduced detection of low-volume stealth attacks and a 42% anomaly precision rate — are documented honestly throughout this report alongside concrete, actionable recommendations for future improvement. This reflects the project's broader goal of producing not just a working model, but a well-understood and extensible one that provides a genuine foundation for a production-grade network anomaly detection capability.

From a learning perspective, the project provided hands-on experience with the complete data science lifecycle: data acquisition from a public repository, data quality challenges (1,358 null values; 4,376 infinite values; column whitespace), feature engineering, unsupervised model selection and justification, hyperparameter calibration from empirical evidence, performance evaluation with multiple metrics, and communicating results through an interpretable dashboard — all skills directly applicable to real-world cybersecurity data science roles.

12. FUTURE SCOPE

Several concrete directions for extending and improving the tool have been identified based on the current system's evaluated performance:

1. Ensemble Detection for Stealth Attacks

The most impactful near-term improvement would be to complement Isolation Forest with a second algorithm targeting the attack categories where it currently underperforms. A One-Class SVM or Local Outlier Factor could be applied specifically to flows whose Isolation Forest anomaly score falls in the intermediate range (neither clearly normal nor clearly anomalous), using their different decision boundary characteristics to resolve borderline cases. This ensemble approach has been shown in the literature to improve detection rates on stealth attacks without significantly increasing false positives.

2. Per-Attack-Type Contamination Tuning

The current implementation uses a single global contamination parameter (0.17) across all 14 attack categories. The per-attack-type detection rates already measured in this project (ranging from <5% to 100%) could be used to calibrate separate thresholds for different traffic sub-populations — for example, applying a more aggressive threshold to high-speed flows (which are more likely to be DoS attacks) and a less aggressive threshold to low-speed flows (which overlap with legitimate sessions more often).

3. Real-Time Streaming Detection

The trained model and scaler are serialised and directly reusable without modification. Extending the system to real-time operation requires wrapping them in a streaming inference loop: live packets would be captured (using Scapy or a NetFlow/IPFIX collector), aggregated into 78-feature flow records using a sliding time window, scaled with the frozen StandardScaler, and classified by the frozen Isolation Forest in near real time. A lightweight FastAPI REST endpoint would allow existing SIEM tools (Splunk, IBM QRadar) to submit flow records and receive anomaly scores.

4. Tiered Alerting and Incident Response Integration

The continuous anomaly score already produced by score_samples() can be directly used for tiered severity classification without additional modelling: scores in the lowest 5th percentile (most anomalous) proposed as Critical, the next 10th percentile as High, remaining flagged anomalies as Medium. High and Critical severity detections would feed into a ticketing system (Jira Service Management or a SOC case management tool) via webhook, with each ticket pre-populated with the relevant flow's feature values and anomaly score to accelerate analyst triage.

5. Scheduled Model Retraining

A rolling-window retraining strategy would minimise model drift: the Isolation Forest would be retrained monthly on the most recent 30 days of confirmed normal traffic, using the same pipeline already implemented. Because training takes only ~20 seconds, this could be triggered automatically on a schedule (cron job or Apache Airflow) or when the running false-positive rate exceeds a defined threshold.

6. Cloud Deployment and Containerisation

The modular pipeline architecture is Docker-ready: each component (ingestion, preprocessing, model, visualisation) can be containerised independently and deployed on cloud infrastructure (IBM Cloud, AWS, Azure) for horizontal scaling. This would extend the tool from a single-developer MVP to an enterprise-grade monitoring service, potentially offered as a SaaS subscription or integrated with MSSPs (Managed Security Service Providers).

7. Feature Engineering Refinement

Packet-length-related features (backward packet length statistics, packet length mean/std/variance) were consistently the strongest anomaly predictors in the EDA correlation analysis (0.5–0.6 correlation magnitude). Future feature engineering effort would be most productive if focused on refining these statistics further — for example, computing rolling statistics over a sliding time window rather than per-flow aggregates — rather than adding entirely new feature categories.

13. APPENDIX

Source Code

The complete project source code is implemented as a single Google Colaboratory notebook:

File: Network_Anomaly_Detection_Project_Code.ipynb

The notebook is structured as follows:

11. Environment setup and library imports (pandas, numpy, sklearn, matplotlib, seaborn, joblib).
12. Data ingestion: loading all eight CICIDS 2017 CSV files from Google Drive and concatenating into a single dataframe (2,830,743 rows, 79 columns).
13. Data cleaning: column whitespace stripping, infinite value replacement, null row removal, duplicate removal — resulting in 2,520,798 clean rows.
14. Feature engineering: binary label creation (Label_binary), feature/label split, StandardScaler fitting and transformation.
15. Exploratory Data Analysis: six-panel visualisation figure (eda_visualizations.png) covering label distribution, normal/attack ratio, flow duration, packet length, top correlated features, and flow bytes/second.
16. Training set preparation: normal-traffic-only subset (2,095,057 rows) for unsupervised Isolation Forest training.
17. Model training: IsolationForest(n_estimators=100, contamination=0.17, random_state=42, n_jobs=-1).fit(X_train), serialised to isolation_forest_model.pkl.
18. Prediction: model.predict(X_test) on full 2,520,798-row test set; binary label conversion.
19. Evaluation: sklearn classification_report and confusion_matrix against ground-truth Label_binary.
20. Results export: anomaly_results.csv containing Actual_Label, Predicted_Label, Anomaly_Score, Traffic_Type for all test rows.
21. Dashboard generation: four-panel anomaly_dashboard.png (anomaly score distribution, confusion matrix heatmap, detection rate by attack type, performance metrics bar chart).

GitHub & Project Demo Link

GitHub Repository:

https://github.com/SreeSai1505/Network-Anomaly-Detection

The repository contains:

- Network_Anomaly_Detection_Project_Code.ipynb — complete Jupyter notebook with all pipeline stages.
- README.md — project overview, setup instructions, and usage guide.
- All project phase documents (Project Design Phase-I, Phase-II, Project Planning, Project Manual).

Developer: Sree Sai | Institution: MIT Manipal — School of Computer Engineering

Programme: SmartBridge / IBM Cyber Security Analyst Virtual Internship (SWAYAM Plus)

Date of Submission: 30 June 2026